

Datenstrukturen, Algorithmen und Programmierung 1

Vorlesung 25

Johannes Fischer

TU Dortmund

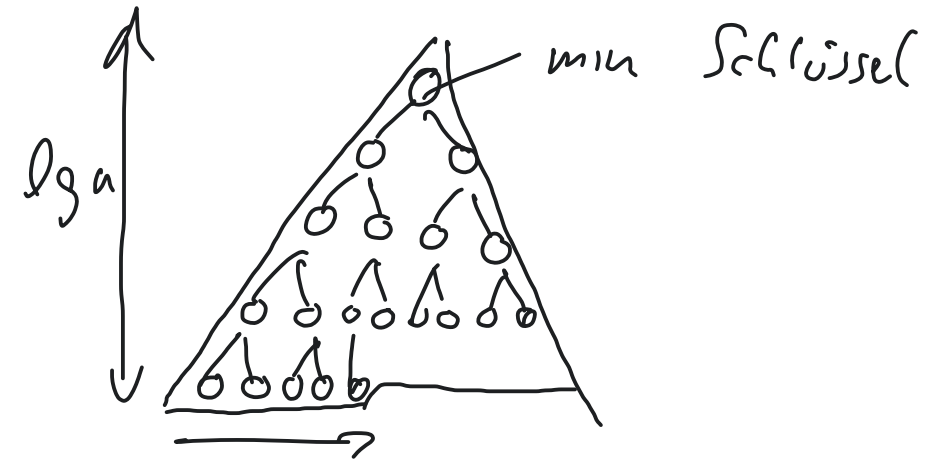
Heaps und Priority Queues

ADT Priority Queue (“Prioritätswarteschlange”)

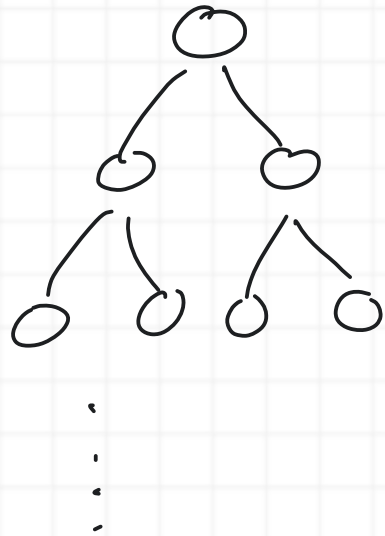
- Eine **Prioritätswarteschlange** (engl. **priority queue**, “PQ”) mit Schlüsseln vom Typ K und Werten vom Typ V hat folgende Operationen:
 - $init(pq_name, capacity, K, V)$: initialisiert eine leere PQ mit Namen pq_name für bis zu $capacity$ viele key-value-Paare vom Typ (K, V)
 - $push(pq_name, \langle k, v \rangle)$: fügt den Wert v (vom Typ V) mit Schlüssel k (vom Typ K) in pq_name ein, falls die Kapazität noch nicht erschöpft ist
 - $extract_min(pq_name)$: löscht ein key-value-Paar $\langle k, v \rangle$ mit **minimalem** Schlüsselwert k aus der PQ und gibt **dessen Wert** zurück ($k = \min\{q : \langle q, w \rangle \in pq_name\}$)
 - $is_empty(pq_name)$: gibt wahr zurück gdw. pq_name leer ist
 - $size(pq_name)$: gibt die Anzahl der key-value-Paare in pq_name zurück
- Wie eine **Queue**, aber pop liefert die Elemente aufsteigend geordnet nach “Priorität”
 - analog: **Maximum-PQs** (absteigend geordnet)

Heaps

- ein **Min-Heap** (dt. Haufen/Halde) ist ein binärer Baum B für n key-value-Paare mit den 2 folgenden Eigenschaften:
 1. **Heap-Ordnung:** der Schlüssel aller Knoten v (außer der Wurzel) ist **nicht kleiner** als der Schlüssel ihres Elters
 2. **Vollständigkeit:** der Baum B ist **vollständig**, d.h. alle Tiefen (bis auf die letzte) sind voll besetzt, und die letzte Tiefe ist “von links nach rechts” besetzt
- Konsequenz: minimaler Schlüssel an Wurzel
- Heaps haben eine Höhe von $\lfloor \log n \rfloor$



Heaps ~~Ar.~~ mit n Knoten haben Höhe $\lfloor \lg n \rfloor$



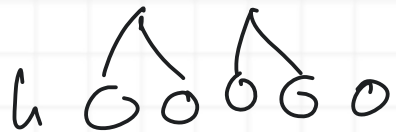
$$\leftarrow 2^0$$

$$\leftarrow 2^1$$

$$\leftarrow 2^2$$

⋮

$$h-1 \quad 0 \quad 0 \quad \dots \quad 0 \quad \leftarrow 2^{h-1}$$



$$n = \sum_{i=0}^{h-1} 2^i + 1 \leq 2^h \Rightarrow n \leq \sum_{i=0}^h 2^i = 2^{h+1} - 1$$

$$n \geq 2^h \quad | \cdot \lg$$

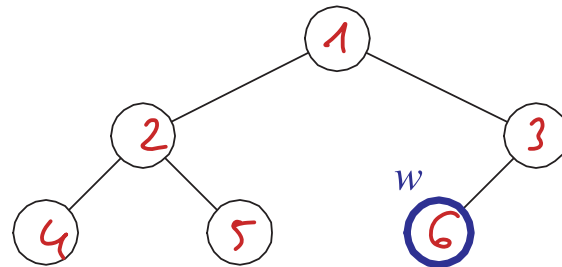
$$\lg n \geq h$$

$$= 2^{h+1} - 1 + 1 = 2^{h+1}$$

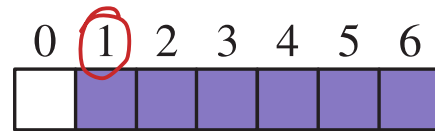
$$\Rightarrow \lg(n+1) \leq h+1$$

$$h \geq \lg(n+1) - 1$$

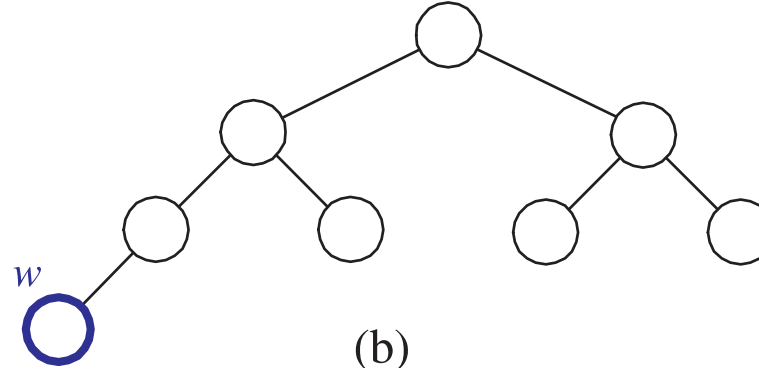
Array-Darstellung von Heaps



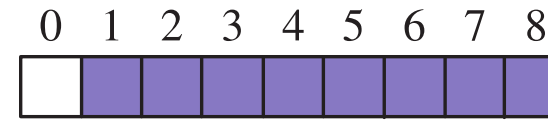
(a)



(c)



(b)



(d)

© Goodrich et al.: Data Structures and Algorithms in C++ (2nd ed.). John Wiley and Sons, 2011.

- “implizite Datenstruktur”

- Elter von Knoten k an Position $\lfloor k/2 \rfloor$
- Kinder von k an Positionen $2k$ und $2k + 1$ (falls vorhanden, d.h. $\leq n$)

Heap-Implementierung

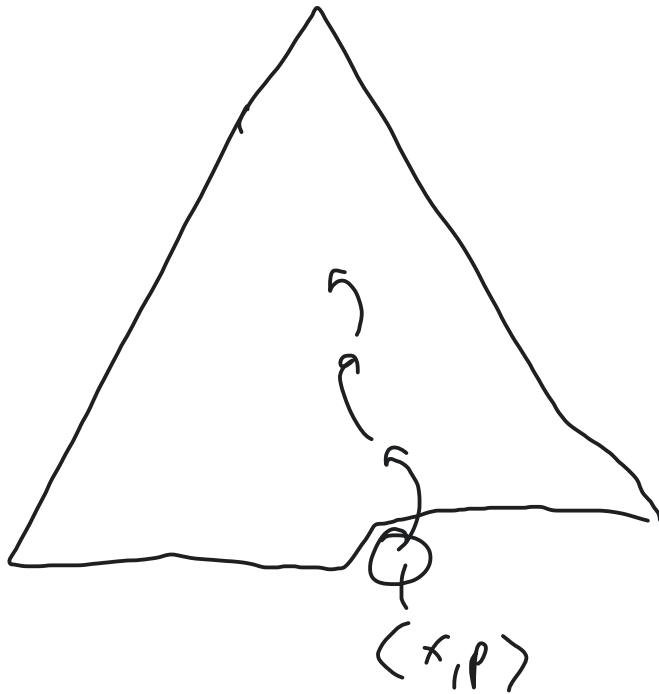
```
namespace dap1 {  
template <typename K, typename V>  
class PQ {  
    struct KVPair { // private "innere Klasse"  
        K key_;  
        V value_;  
  
        KVPair(K const& key = {}, V const& val = {}) : key_{key}, value_{val} {}  
  
        // implement operators to use keys:  
        bool operator<(KVPair const& other) const { return key_ < other.key_; }  
        bool operator==(KVPair const& other) const { return key_ == other.key_; }  
    };  
  
    KVPair* array_;  
    int n_;  
  
    // ...  
};
```

- NB: KVPair ist eine **innere Klasse** von PQ (ansonsten wie bei Map)
 - d.h. kann nur innerhalb von PQ benutzt werden
 - ist generisch (Typen U, V), aber innerhalb von PQ ohne Template-Parameter benutzbar (KVPair und nicht KVPair<U, V>)

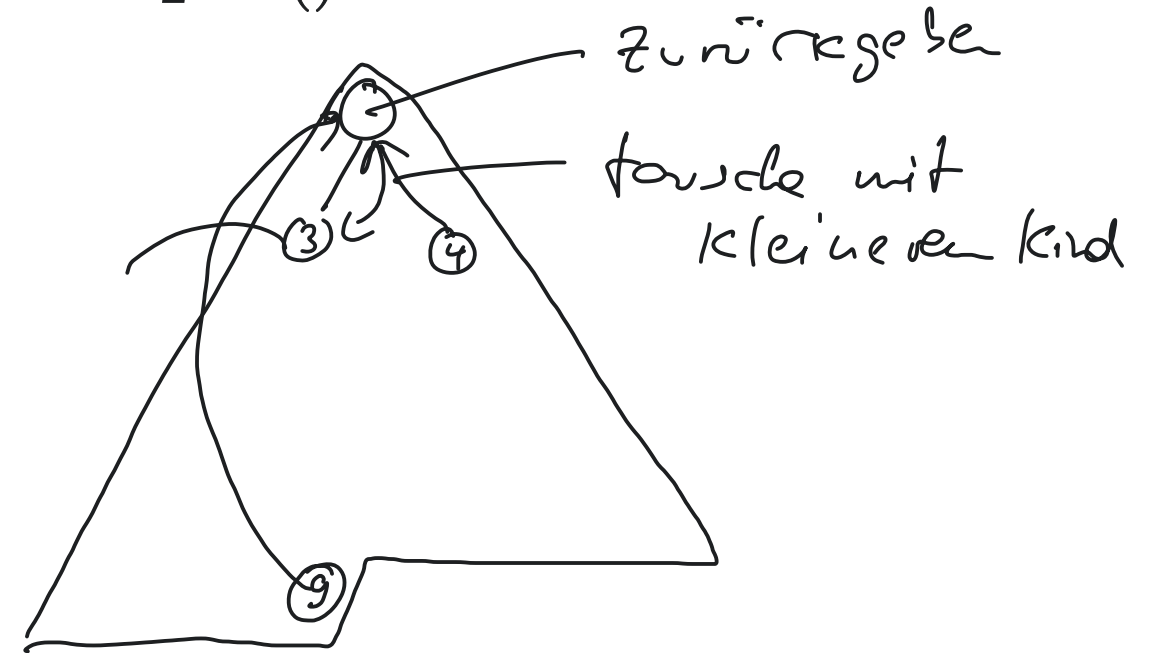
Heap-Operationen

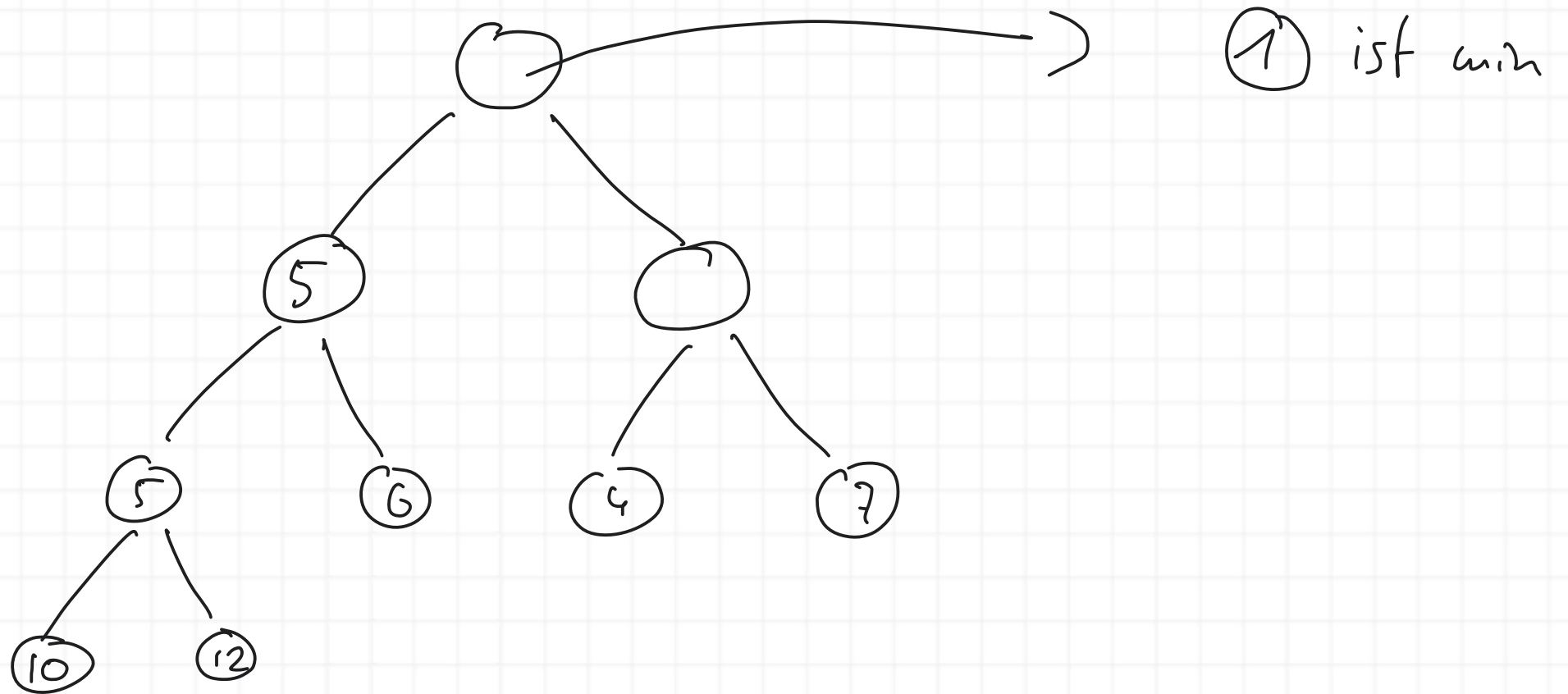
- durch die Höhe $\lceil \log n \rceil$ implementieren Heaps den ADT PQ schnell:

$push(\langle x, p \rangle)$



$extract_min()$





PQ-Operationen mit Heaps in C++

```
1 public:
2   PQ(int capacity) : array_{new KVPair[capacity + 1]}, n_{0} {} // Konstruktor
```

```
1 public:
2   void push(K const& k, V const& v) {
3     // erzeuge "neuen Knoten" als neues letztes Blatt:
4     array_[++n_] = KVPair(k, v);
5     // jetzt Heap-Bedingung wiederherstellen:
6     sift_up(n_);
7   }
8 private:
9   void sift_up(int k) {
10    while (k > 1 && array_[k] < array_[k / 2]) {
11      swap(k, k / 2);
12      k = k / 2;
13    }
14  }
15
16  // swap array_[i] with array_[j]
17  void swap(int i, int j) {
18    T tmp = array_[i];
19    array_[i] = array_[j];
20    array_[j] = tmp;
21  }
```

```
1 public:
2   V extract_min() {
3     KVPair result = array_[1];
4     array_[1] = array_[n_];
5     --n_;
6     sift_down(1); // vertausche nach unten, bis Heap-Bed gilt
7     return result.value_;
8   }
9 private:
10  void sift_down(int k) {
11    while (2 * k < n_) {
12      int j = 2 * k; // left child of k
13      if (j < n_ - 1 && array_[j + 1] < array_[j])
14        ++j; // goto right child if smaller
15      if (array_[k] <= array_[j]) return;
16      swap(k, j);
17      k = j; // continue at child
18    }
19  }
```

Sortieren mit PQs/Heaps

- füge alle Paare in eine Min-PQ ein und lösche sie anschließend in aufsteigend sortierter Reihenfolge, z.B. sortiere Studierende nach Matr.-Nummern:

```
1  int n = 100;
2  dap1::PQ<int, Stud> pq(n);
3
4  std::random_device rd; // create 100 students with random id (just testing)
5  for (int i = 0; i < n; ++i) {
6      int nr = rd() % 1000000;
7      pq.push(nr, Stud{nr, 25, 'd'});
8  }
9
10 Stud* sorted_students = new Stud[n]; // hier werden die Stud. sortiert eingefügt
11 int i = 0;
12 while (!pq.is_empty()) {
13     Stud minimum = pq.extract_min();
14     sorted_students[i++] = minimum;
15 }
```

Da die Höhe des entstehenden Heaps max. $\lceil \log n \rceil$ ist, sortiert Heap-Sort ein Array der Länge n mit $C \cdot n \log n$ Operationen.

Iteratoren

Iteratoren

- Die C++-STL hat (wie viele Programmiersprachen) ein Iteratorenkonzept für Container
 - **Container**: generische Datenstrukturen zum Speichern einer Sammlung von Objekten (z.B. `ResizableArray<T>`, `MyStack<T>`, `BST<T>`, ...)
- Zugriff auf die einzelnen Objekte bisher:
 - mit `[]` in `MyArray<T>`, `ResizableArray<T>`, `Map<K, V>`
 - mit `pop()` in `MyStack<T>`
 - usw.
- Iteratoren: einheitliche Schnittstelle zum Zugriff auf alle Objekte des Containers:
 - mit `begin()` erhält man das erste Element, `end()` zeigt auf das Element direkt **nach** dem letzten
 - `++` geht zum nächsten Element
 - `*` dereferenziert das Element, d.h. gibt eine Referenz auf ein `T`-Objekt zurück

- != vergleicht 2 Iteratoren

Am Beispiel von Heaps

```
1  template <typename T> class PQ {
2      T* array_;
3      int n_;
4
5  public:
6      class Iterator { // an iterator for the pq
7          T* arr_;
8          int current_;
9
10     public:
11         Iterator(T* arr = nullptr, int i = 0) : arr_{arr}, current_{i} {}
12
13         T& operator*() {
14             return arr_[current_]; // reference to the element
15         }
16
17         Iterator& operator++() {
18             ++current_;
19             return *this;
20         }
21         bool operator!=(Iterator const& p) {
22             return arr_ != p.arr_ || current_ != p.current_;
23         }
24     };
25
26     Iterator begin() { return Iterator(array_, 1); }
27     Iterator end() { return Iterator(array_, n_ + 1); }
28 }
```

```
1  int main() {
2      int n = 100;
3
4      dap1::PQ<int> pq(n);
5
6      for (int i = 0; i < n; ++i) pq.push(rand() % 100);
7
8      for (dap1::PQ<int>::Iterator it = pq.begin();
9           it != pq.end(); ++it) {
10         printf("%i,", *it);
11     }
12     printf("\n");
13
14
15
16
17     // äquivalent mit "range based for":
18     for (int i : pq) { // wird übersetzt wie oben
19         printf("%i,", i);
20     }
21     printf("\n");
22 }
```

Am Beispiel von Linked List

```
1  template <typename T>
2  class LinkedList {
3      Node<T>* head_; // Zeiger auf den Anfang der LL
4      Node<T>* last_; // Zeiger auf das Ende der LL
5      int size_;      // Anzahl der Elemente in LL
6
7  public:
8      class Iterator { // an iterator for the LL
9          Node<T>* current_;
10
11     public:
12         Iterator(Node<T>* c = nullptr) : current_{c} {}
13
14         T& operator*() {
15             return current_->cont_; // reference to the element
16         }
17
18         Iterator& operator++() {
19             current_ = current_->next_;
20             return *this;
21         }
22
23         bool operator!=(Iterator const& p) { return current_ != p.current_; }
24     };
25     Iterator begin() { return Iterator(head_); }
26     Iterator end() { return Iterator(nullptr); } // correct since last_->next = nullptr
27
28     // ...
```

- NB: alle Aspekte rund um `const` hier nicht beachtet

Iterator für ResizableArray

- (s. VSCode für die const-Variante)

